Lab Assignment 6

```
!pip install mrjob
```

```
Requirement already satisfied: mrjob in /usr/local/lib/python3.12/dist-packages (0.7.4)
Requirement already satisfied: PyYAML>=3.10 in /usr/local/lib/python3.12/dist-packages (from mrjob) (6.0.3)
```

```
# Define  Shuffle and Average Reducer
from collections import defaultdict

def manual_shuffle(mapped_data):
    shuffled = defaultdict(list)
    for key, value in mapped_data:
        shuffled[key].append(value)
    return shuffled.items()

def avg_reducer(shuffled_data):
    for key, values in shuffled_data:
        yield (key, sum(values) / len(values))
```

Q1. Word Count

Input: `hadoop is fast \n hadoop is scalable`

```
print("--- (A) Manual Approach ---")
q1_data = ["hadoop is fast", "hadoop is scalable"]

def q1_mapper(data):
    for line in data:
        for word in line.split():
            yield (word, 1)

def sum_reducer(shuffled_data):
    for key, values in shuffled_data:
        yield (key, sum(values))

q1_mapped = list(q1_mapper(q1_data))
q1_shuffled = manual_shuffle(q1_mapped)
q1_reduced = list(sum_reducer(q1_shuffled))
print(q1_reduced)
```

```
--- (A) Manual Approach ---
[('hadoop', 2), ('is', 2), ('fast', 1), ('scalable', 1)]
```

```
%%writefile q1_mrjob.py
from mrjob.job import MRJob
import re

class MRWordCount(MRJob):
    def mapper(self, _, line):
        for word in re.findall(r'\w+', line.lower()):
            yield (word, 1)

    def reducer(self, word, counts):
        yield (word, sum(counts))

if __name__ == '__main__':
    MRWordCount.run()
```

```
Overwriting q1_mrjob.py
```

```
print("--- (B) mrjob Framework ---")
!echo "hadoop is fast\nhadoop is scalable" > q1_input.txt
!python q1_mrjob.py q1_input.txt 2> /dev/null
```

```
--- (B) mrjob Framework ---
"is"    2
"nhadoop"       1
"scalable"      1
"fast"  1
"hadoop"        1
```

Q2. Character Count (Ignore Spaces)

Input: `big data`

```
print("--- (A) Manual Approach ---")
q2_data = ["big data"]

def q2_mapper(data):
    for line in data:
        for char in line.replace(" ", ""):
            yield (char, 1)

q2_mapped = list(q2_mapper(q2_data))
q2_shuffled = manual_shuffle(q2_mapped)
q2_reduced = list(sum_reducer(q2_shuffled))
print(q2_reduced)
```

```
--- (A) Manual Approach ---
[('b', 1), ('i', 1), ('g', 1), ('d', 1), ('a', 2), ('t', 1)]
```

```
%%writefile q2_mrjob.py
from mrjob.job import MRJob

class MRCharCount(MRJob):
    def mapper(self, _, line):
        for char in line.replace(" ", ""):
            yield (char, 1)

    def reducer(self, char, counts):
        yield (char, sum(counts))

if __name__ == '__main__':
    MRCharCount.run()
```

```
Overwriting q2_mrjob.py
```

```
print("--- (B) mrjob Framework ---")
!echo "big data" > q2_input.txt
!python q2_mrjob.py q2_input.txt 2> /dev/null
```

```
--- (B) mrjob Framework ---
"b"     1
"d"     1
"g"     1
"i"     1
"t"     1
"a"     2
```

Q3. Average Word Length (Per Word)

Input: `data science data big data`

```
print("--- (A) Manual Approach ---")
q3_data = ["data science data big data"]

def q3_mapper(data):
    import re
    for line in data:
        for word in re.findall(r'\w+', line.lower()):
            yield (word, len(word))

q3_mapped = list(q3_mapper(q3_data))
q3_shuffled = manual_shuffle(q3_mapped)
q3_reduced = list(avg_reducer(q3_shuffled))
print(q3_reduced)
```

```
--- (A) Manual Approach ---
[('data', 4.0), ('science', 7.0), ('big', 3.0)]
```

```
%%writefile q3_mrjob.py
from mrjob.job import MRJob
import re

class MRPerWordAvg(MRJob):
    def mapper(self, _, line):
        for word in re.findall(r'\w+', line.lower()):
            yield (word, len(word))
```

```
    def reducer(self, word, lengths):
        l = list(lengths)
        yield (word, sum(l) / len(l))

if __name__ == '__main__':
    MRPerWordAvg.run()
```

```
Overwriting q3_mrjob.py
```

```
print("--- (B) mrjob Framework ---")
!echo "data science data big data" > q3_input.txt
!python q3_mrjob.py q3_input.txt 2> /dev/null
```

```
--- (B) mrjob Framework ---
"science"       7.0
"big"   3.0
"data"  4.0
```

Q4. Global Average Word Length

```
print("--- (A) Manual Approach ---")
q4_data = ["hadoop mapreduce spark"]

def q4_mapper(data):
    import re
    for line in data:
        for word in re.findall(r'\w+', line.lower()):
            yield ("global_avg", len(word))

q4_mapped = list(q4_mapper(q4_data))
q4_shuffled = manual_shuffle(q4_mapped)
q4_reduced = list(avg_reducer(q4_shuffled))
print(q4_reduced)
```

```
--- (A) Manual Approach ---
[('global_avg', 6.666666666666667)]
```

```
%%writefile q4_mrjob.py
from mrjob.job import MRJob
import re

class MRGlobalAvg(MRJob):
    def mapper(self, _, line):
        for word in re.findall(r'\w+', line):
            yield ("global_avg", len(word))

    def reducer(self, key, lengths):
        l = list(lengths)
        yield (key, sum(l) / len(l))

if __name__ == '__main__':
    MRGlobalAvg.run()
```

```
Overwriting q4_mrjob.py
```

```
print("--- (B) mrjob Framework ---")
!echo "hadoop mapreduce spark" > q4_input.txt
!python q4_mrjob.py q4_input.txt 2> /dev/null
```

```
--- (B) mrjob Framework ---
"global_avg"    6.666666666666667
```

Q5. External Dataset

```
import os

Q5_FILE = "q5_dataset.txt"

if os.path.exists(Q5_FILE):
    print(f"Found {Q5_FILE}! Proceeding...")
    print("\n--- First 5 lines of the dataset ---")
    !head -n 5 q5_dataset.txt
else:
    print(f"WARNING: '{Q5_FILE}' not found! Please upload the file to Colab and rename it to '{Q5_FILE}' before continuing.
```

```
Found q5_dataset.txt! Proceeding...

--- First 5 lines of the dataset ---
The Project Gutenberg EBook of The Complete Works of William Shakespeare, by
William Shakespeare

This eBook is for the use of anyone anywhere at no cost and with
almost no restrictions whatsoever.  You may copy it, give it away or
```

```
%%writefile q5_manual_runner.py
import re
from collections import defaultdict

def manual_shuffle(mapped_data):
    shuffled = defaultdict(list)
    for key, value in mapped_data:
        shuffled[key].append(value)
    return shuffled.items()

print("\n--- (A) Running Manual Approach for Q1-Q4 on Q5 Dataset ---")
q1_map, q2_map, q3_map, q4_map = [], [], [], []

try:
    with open('q5_dataset.txt', 'r', encoding='utf-8') as f:
        for line in f:
            # Mapper logic
            words = re.findall(r'[a-zA-Z]+', line.lower())
            for w in words:
                q1_map.append((w, 1))
                q3_map.append((w, len(w)))
                q4_map.append(("global_avg", len(w)))
            for char in re.findall(r'[a-z]', line.lower()):
                q2_map.append((char, 1))

    # Reducer logic (Taking top 10 items to save screen space)
    q1_res = sorted([(k, sum(v)) for k, v in manual_shuffle(q1_map)], key=lambda x: -x[1])[:10]
    q2_res = sorted([(k, sum(v)) for k, v in manual_shuffle(q2_map)], key=lambda x: -x[1])[:10]
    q3_res = sorted([(k, sum(v)/len(v)) for k, v in manual_shuffle(q3_map)])[:10]
    q4_res = [(k, sum(v)/len(v)) for k, v in manual_shuffle(q4_map)]

    print("\nQ1 Word Count (Top 10):", q1_res)
    print("\nQ2 Char Count (Top 10):", q2_res)
    print("\nQ3 Word Lengths (First 10 distinct):", q3_res)
    print("\nQ4 Global Avg Length:", q4_res)
except FileNotFoundError:
    print("Please upload q5_dataset.txt first.")
```

```
Overwriting q5_manual_runner.py
```

```
!python q5_manual_runner.py
```

```
--- (A) Running Manual Approach for Q1-Q4 on Q5 Dataset ---

Q1 Word Count (Top 10): [('the', 27843), ('and', 26847), ('i', 22538), ('to', 19882), ('of', 18307), ('a', 14800), ('you', 1

Q2 Char Count (Top 10): [('e', 448860), ('t', 331121), ('o', 315828), ('a', 290069), ('i', 255023), ('s', 249687), ('n', 244

Q3 Word Lengths (First 10 distinct): [('a', 1.0), ('aaron', 5.0), ('abaissiez', 9.0), ('abandon', 7.0), ('abandoned', 9.0),

Q4 Global Avg Length: [('global_avg', 4.087839420355874)]
```

```
print("\n--- (B) Running mrjob Framework Q1-Q4 on Q5 Dataset (Logs Muted) ---")

print("\n[Q1 mrjob Output - Top 10]")
!python q1_mrjob.py q5_dataset.txt > q5_q1_output.txt 2> /dev/null
!head -n 10 q5_q1_output.txt

print("\n[Q2 mrjob Output - Top 10]")
!python q2_mrjob.py q5_dataset.txt > q5_q2_output.txt 2> /dev/null
!head -n 10 q5_q2_output.txt

print("\n[Q3 mrjob Output - Top 10]")
!python q3_mrjob.py q5_dataset.txt > q5_q3_output.txt 2> /dev/null
!head -n 10 q5_q3_output.txt

print("\n[Q4 mrjob Output]")
!python q4_mrjob.py q5_dataset.txt 2> /dev/null
```

```
--- (B) Running mrjob Framework Q1-Q4 on Q5 Dataset (Logs Muted) ---
```

```
[Q1 mrjob Output - Top 10]
"faced" 2
"facere"        1
"faces" 53
"faciant"       1
"facile"        1
"facility"      4
"facinerious"   1
"facing"        1
"facit" 2
"fact"  12

[Q2 mrjob Output - Top 10]
"c"     67194
"d"     134216
"e"     406157
"f"     69103
"g"     57328
"h"     218875
"i"     199130
"j"     2788
"k"     29345
"l"     146532

[Q3 mrjob Output - Top 10]
"eyesight"      8.0
"eyestrings"    10.0
"eying" 5.0
"eyne"  4.0
"eyrie" 5.0
"f"     1.0
"fa"    2.0
"fabian"        6.0
"fable" 5.0
"fables"        6.0

[Q4 mrjob Output]
"global_avg"    4.084902057952173
```

```
%%writefile q5_top5_mrjob.py
from mrjob.job import MRJob
from mrjob.step import MRStep
import re

class MRTop5Words(MRJob):
    def steps(self):
        return [
            MRStep(mapper=self.mapper_get_words,
                   combiner=self.combiner_count_words,
                   reducer=self.reducer_count_words),
            MRStep(reducer=self.reducer_find_top_5)
        ]

    def mapper_get_words(self, _, line):
        # Extract pure alphabet words only, ignoring punctuation and code garbage
        for word in re.findall(r'[a-zA-Z]+', line):
            clean_word = word.lower()
            if len(clean_word) < 20:
                yield (clean_word, 1)

    def combiner_count_words(self, word, counts):
        yield (word, sum(counts))

    def reducer_count_words(self, word, counts):
        # Yield None as key so all pairs go to a single reducer for global sorting
        yield None, (sum(counts), word)

    def reducer_find_top_5(self, _, word_count_pairs):
        # Sort descending by count and take the top 5
        top5 = sorted(word_count_pairs, reverse=True)[:5]
        for count, word in top5:
            yield (word, count)

if __name__ == '__main__':
    MRTop5Words.run()
```

```
Overwriting q5_top5_mrjob.py
```

```
print("\n--- Finding Top 5 Most Frequent Words in Q5 Dataset ---")
!python q5_top5_mrjob.py q5_dataset.txt 2> /dev/null
```

```

--- Finding Top 5 Most Frequent Words in Q5 Dataset ---
"the"   27843
"and"   26847
```

```
"i"     22538
"to"    19882
"of"    18307
```

Q6. Compute Average Marks for Each Student

```
print("--- (A) Manual Approach ---")
q6_data = ["A 80", "B 70", "A 90", "B 60", "A 100"]

def q6_mapper(data):
    for line in data:
        student, mark = line.split()
        yield (student, int(mark))

q6_mapped = list(q6_mapper(q6_data))
q6_shuffled = manual_shuffle(q6_mapped)
q6_reduced = list(avg_reducer(q6_shuffled))
print(q6_reduced)
```

```
--- (A) Manual Approach ---
[('A', 90.0), ('B', 65.0)]
```

```
%%writefile q6_mrjob.py
from mrjob.job import MRJob

class MRAvgMarks(MRJob):
    def mapper(self, _, line):
        try:
            student, mark = line.split()
            yield (student, int(mark))
        except ValueError:
            pass

    def reducer(self, student, marks):
        m = list(marks)
        yield (student, sum(m) / len(m))

if __name__ == '__main__':
    MRAvgMarks.run()
```

```
Overwriting q6_mrjob.py
```

```
print("\n--- (B) mrjob Framework ---")
!echo -e "A 80\nB 70\nA 90\nB 60\nA 100" > q6_input.txt
!python q6_mrjob.py q6_input.txt 2> /dev/null
```

```

--- (B) mrjob Framework ---
"B"     65.0
"A"     90.0
```

Q7. Average Salary per Department & Highest Paid

```
print("--- (A) Manual Approach ---")
q7_data = ["HR 50000", "IT 70000", "HR 60000", "IT 80000"]

def q7_mapper(data):
    for line in data:
        dept, salary = line.split()
        yield (dept, int(salary))

q7_mapped = list(q7_mapper(q7_data))
q7_shuffled = manual_shuffle(q7_mapped)
dept_avgs = [(k, sum(v)/len(v)) for k, v in q7_shuffled]

print("Department Averages:", dept_avgs)
highest = max(dept_avgs, key=lambda x: x[1])
print(f"Highest Paid Department: {highest[0]} with {highest[1]}")
```

```
--- (A) Manual Approach ---
Department Averages: [('HR', 55000.0), ('IT', 75000.0)]
Highest Paid Department: IT with 75000.0
```

```
%%writefile q7_mrjob.py
from mrjob.job import MRJob
from mrjob.step import MRStep
```

```
class MRHighestAvgSalary(MRJob):
    def steps(self):
        return [
            MRStep(mapper=self.mapper_dept, reducer=self.reducer_avg),
            MRStep(reducer=self.reducer_max)
        ]

    def mapper_dept(self, _, line):
        try:
            dept, salary = line.split()
            yield (dept, int(salary))
        except ValueError:
            pass

    def reducer_avg(self, dept, salaries):
        s = list(salaries)
        avg_salary = sum(s) / len(s)
        # Yield an identical key so all data aggregates in the final reducer
        yield "ALL_DEPTS", (avg_salary, dept)

    def reducer_max(self, _, avg_salaries):
        salaries_list = list(avg_salaries)

        # Print all individual department averages
        for avg, dept in salaries_list:
            yield (f"Average for {dept}:", avg)

        # Print the highest
        highest = max(salaries_list)
        yield ("Highest Paid Department:", highest)

if __name__ == '__main__':
    MRHighestAvgSalary.run()
```

```
Overwriting q7_mrjob.py
```

```
print("\n--- (B) mrjob Framework ---")
!echo -e "HR 50000\nIT 70000\nHR 60000\nIT 80000" > q7_input.txt
!python q7_mrjob.py q7_input.txt 2> /dev/null
```

```
--- (B) mrjob Framework ---
"Average for IT:"       75000.0
"Average for HR:"       55000.0
"Highest Paid Department:"      [75000.0, "IT"]
```

Q8. Average Temperature per Country

```
print("--- (A) Manual Approach ---")
q8_data = ["New York, 38", "London, 29", "Tokyo, 35", "New York, 32", "Delhi, 45", "Ambala, 35"]

def q8_mapper(data):
    for line in data:
        city, temp = line.split(',')
        yield (city.strip(), float(temp.strip()))

q8_mapped = list(q8_mapper(q8_data))
q8_shuffled = manual_shuffle(q8_mapped)
q8_reduced = list(avg_reducer(q8_shuffled))
print(q8_reduced)
```

```
--- (A) Manual Approach ---
[('New York', 35.0), ('London', 29.0), ('Tokyo', 35.0), ('Delhi', 45.0), ('Ambala', 35.0)]
```

```
%%writefile q8_mrjob.py
from mrjob.job import MRJob

class MRAvgTemp(MRJob):
    def mapper(self, _, line):
        parts = line.split(',')
        if len(parts) == 2:
            country, temp = parts[0].strip(), float(parts[1].strip())
            yield (country, temp)

    def reducer(self, country, temps):
        t = list(temps)
        yield (country, sum(t) / len(t))
```

```
if __name__ == '__main__':
    MRAvgTemp.run()
```

```
Overwriting q8_mrjob.py
```

```
print("\n--- (B) mrjob Framework ---")
!echo -e "New York, 38\nLondon, 29\nTokyo, 35\nNew York, 32\nDelhi, 45\nAmbala, 35" > q8_input.txt
!python q8_mrjob.py q8_input.txt 2> /dev/null
```

```
--- (B) mrjob Framework ---
"London"        29.0
"New York"      35.0
"Tokyo" 35.0
"Ambala"        35.0
"Delhi" 45.0
```

Q9. Kaggle Dataset

```
%%writefile q9_kaggle_mrjob.py
from mrjob.job import MRJob
import csv

class MRKaggleAvgTemp(MRJob):
    def mapper(self, _, line):
        for row in csv.reader([line]):

            if len(row) >= 4 and row[1]:
                try:
                    temp = float(row[1])
                    country = row[3].strip()
                    if country != "Country":
                        yield (country, temp)
                except ValueError:
                    pass

    def reducer(self, country, temps):
        t = list(temps)
        yield (country, sum(t) / len(t))

if __name__ == '__main__':
    MRKaggleAvgTemp.run()
```

```
Overwriting q9_kaggle_mrjob.py
```

```
import os

ZIP_FILE = "q9 data.zip"

if os.path.exists(ZIP_FILE):
    print(f"Found {ZIP_FILE}. Unzipping...")
    !unzip -q -o "q9 data.zip" -d q9_unzipped

    print("\nFinding CSV and running MapReduce (this may take a moment for large datasets)...")
    !find q9_unzipped -name "*.csv" -exec python q9_kaggle_mrjob.py {} + > q9_final_output.txt 2> /dev/null

    print("\nDone! Displaying top 20 lines of the output:")
    !head -n 20 q9_final_output.txt
else:
    print(f"WARNING: '{ZIP_FILE}' not found! Please upload the zip file to Colab and run this cell again.")
```

```
Found q9 data.zip. Unzipping...

Finding CSV and running MapReduce (this may take a moment for large datasets)...

Done! Displaying top 20 lines of the output:
"Dalian"        10.229076814988291
"Dar Es Salaam" 25.744671500630517
"Delhi" 25.16586090225564
"Dhaka" 25.490567930489732
"Durban"        20.35129204484784
"Faisalabad"    24.13839654344695
"Fortaleza"     27.008639541892705
"Gizeh" 21.221259213759215
"Guangzhou"     21.60868426103647
"Harare"        20.207842011834316
"Harbin"        3.625744065602072
"Ho Chi Minh City"      27.19398356694055
"Hyderabad"     26.869334928229666
"Ibadan"        26.37310517141197
"Istanbul"      13.507409033480734
```

```
"Izmir" 17.278064578005115
"Jaipur"        25.39305794080867
"Jakarta"       26.508209686003195
"Jiddah"        27.69206601731602
"Jinan" 13.089813819577735
```